Dimensionality Reduction Methods

Background

In everyday bioinformatics study, we are often face with the problem of low number of data (n) and high number of dimensionality (p).

For example:

- Microbiome Sequencing: Low sample number, but usually around 10000 Operational Taxonomic Units
- RNA sequencing: More number of RNA seq than the number of cells
- Genome Sequencing: the number of individuals are less than the number of genes sequences

The curse of dimensionality

- Increasing the number of the dimensions makes it impossible to do regular regression
  - If we have more than 2 data points, it's very easy to fit a linear curve.
  - This is similar to the case where n > p, means that we have more points than the dimensionality
  - However, if we only have one data point, it's impossible to fit a line for.
  - This is similar to the case where n < p, which means that we have too little data points and too many dimensions
- If we have a model fitting all of these dimensions, we will have too many parameters
  - This increases the training time
  - It also increases the chance of overfitting

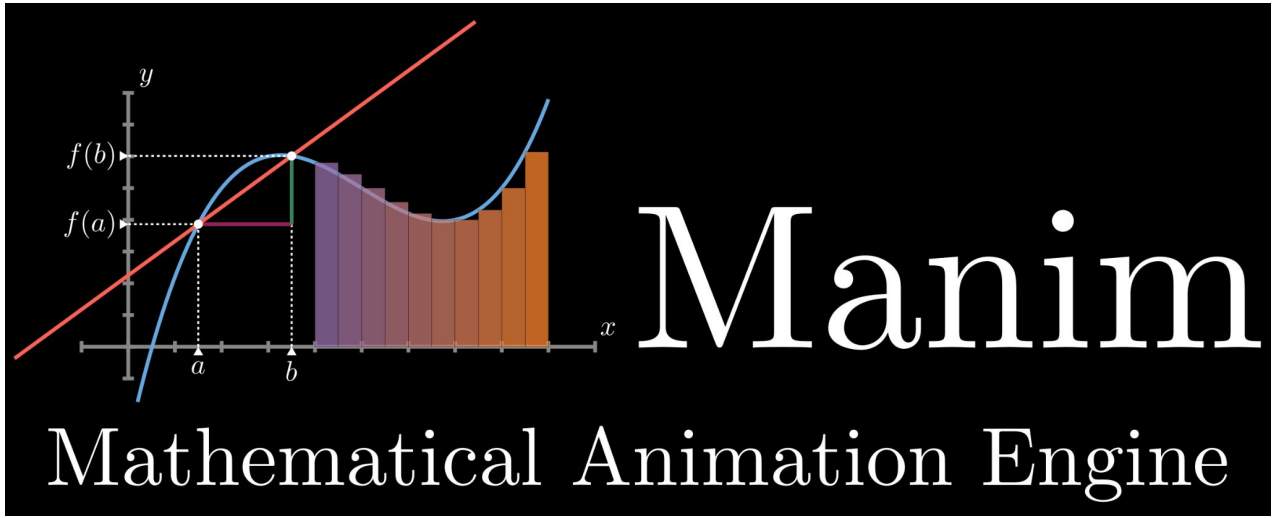
Ways of Reducing the dimensions

- Principal Component Analysis
- tSNE
- UMAP
- MDS
- Autoencoders
- Combinations of previous

Reference

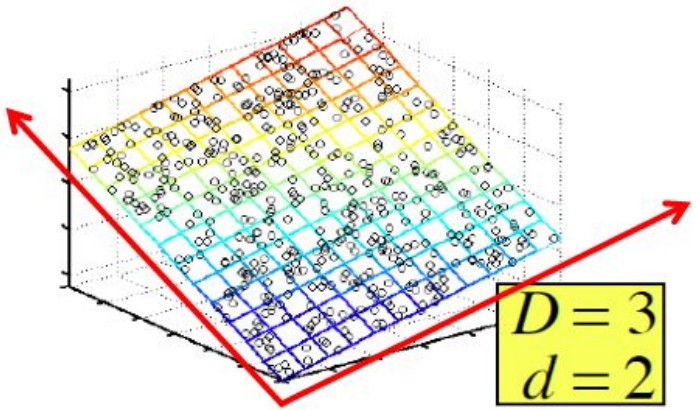
The videos I included in the slides are created by manim, which is a open source mathematical visualization python library open sourced by Grant Sanderson

https://github.com/3b1b/manim



How to interpret the Principle Component Analysis

- The mathematical principle behind the PCA is the singular value decomposition (SVD)
- Intuitive understanding is that we are selecting the direction in space that can capture the highest variance in our dataset



Limit of PCA

It’s difficult to visualize the data is the data pattern is non linear

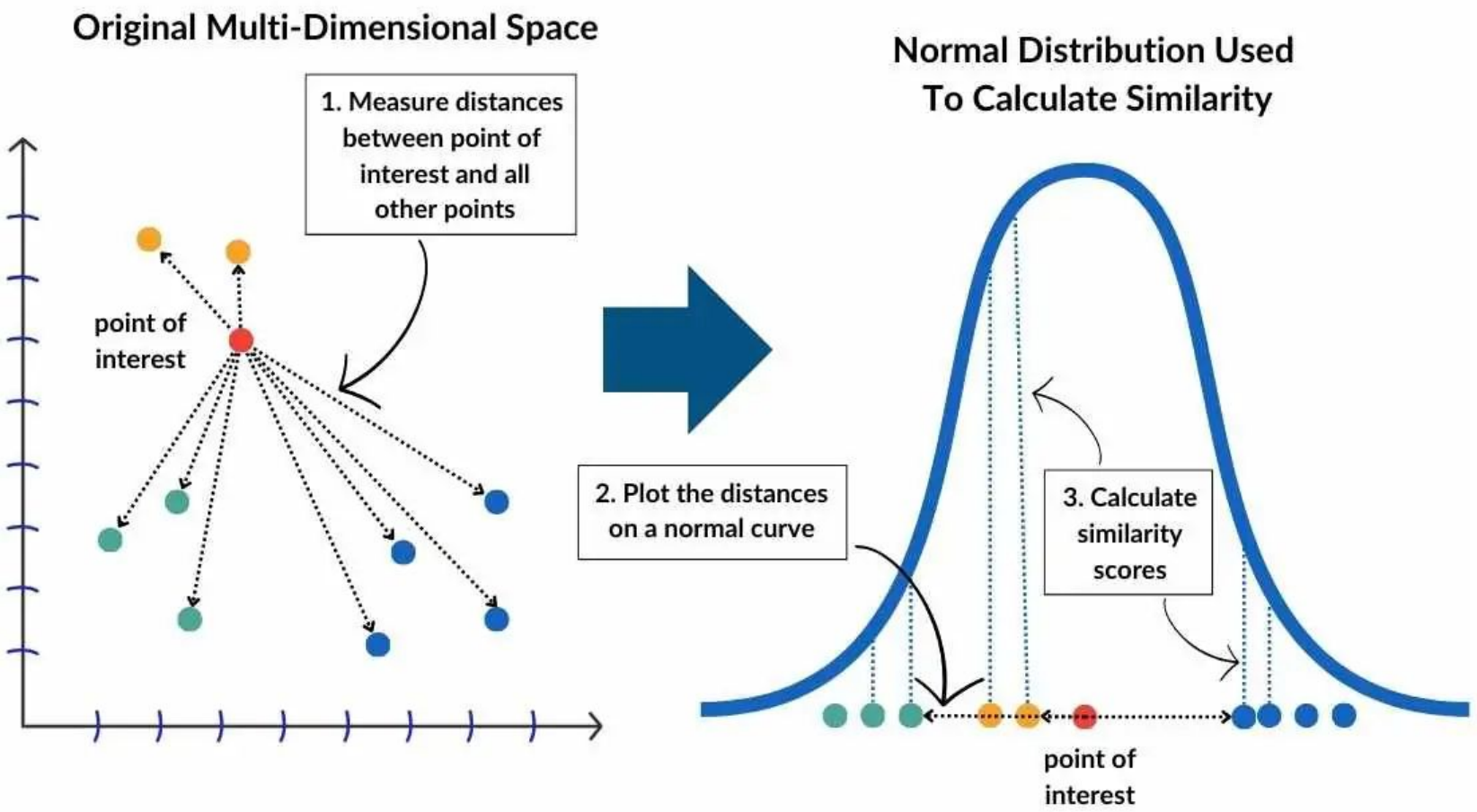
Here is a visualization of the time PCA fails

PCA pros and cons

- PCA is good at
  - Very fast
  - Very easy to interpret the weights
- PCA is not good at
  - Dealing with non-linear data
  - This is simply because PCA only use linear algebra

tSNE

- Stands for t-distributed stochastic neighbor embeddings
- Randomly initialize the points in the low dimension
- Converts the points in the higher dimension into a probability distribution
- Converts the points in the lower dimension also into a probability distribution
- Compare the difference between these 2 distributions, generate a loss function
- Gradient Descent


Original Multi-Dimensional Space
1. Measure distances between point of interest and all other points
point of interest
Normal Distribution Used To Calculate Similarity
2. Plot the distances on a normal curve
3. Calculate similarity scores
point of interest

Point Pairs	(0, 1)	(0, 2)	(0, 3)	(0, 4)
Distance	1	2	3	4

$$\exp(-\|\mathbf{x}_i - \mathbf{x}_j\|^2 / 2\sigma_i^2)$$

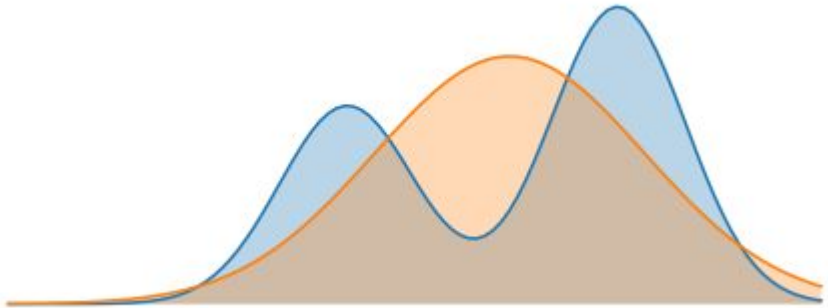
Point Pairs	(0, 1)	(0, 2)	(0, 3)	(0, 4)
Probability	0.9048	0.6703	0.4065	0.2018

We normalize these values
I.e. scale them so they add up to one

Point Pairs	(0, 1)	(0, 2)	(0, 3)	(0, 4)
Probability (normalized)	0.4143	0.3069	0.1861	0.0924

KL-Divergence

- Similarly, we can get a distribution from the lower dimensional representation of the data
- Then we measure the distance between the 2 distribution using the KL-divergence



High Dimensional Distribution

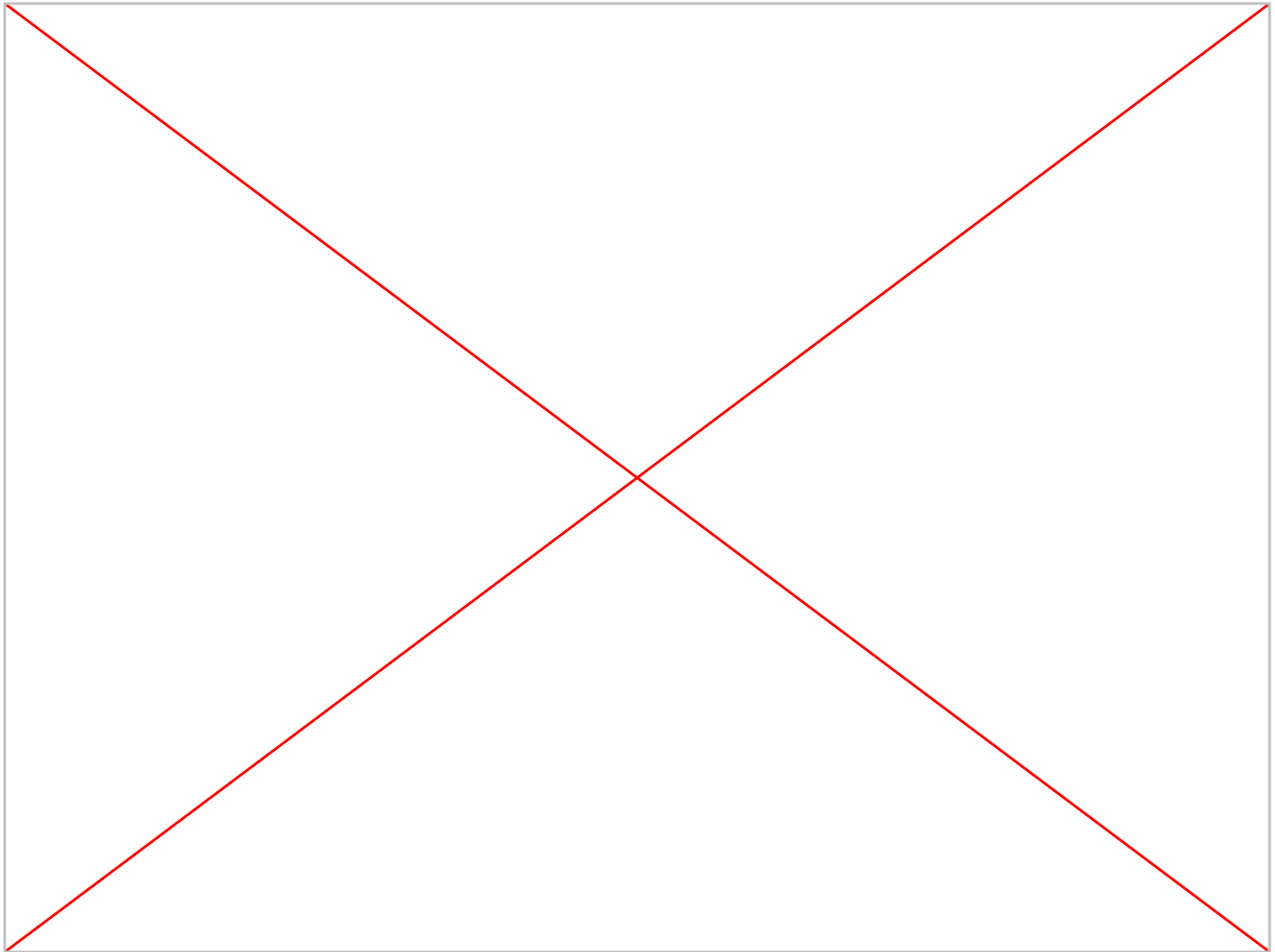
Low Dimensional Distribution

tSNE Pros and Cons

- Pros:
  - Can be used to classify the non-linear data
- Cons:
  - Very slow
  - The hyperparameter perplexity is hard to tune
  - Changing the perplexity completely changes the pattern of the latent space representation

UMAP

- Stands for Uniform Manifold Approximation and Projection
- The math behind it relies on topology
- However the logic is similar
- We start by randomly assigning the location of the points, constructing a graph, the weights are which are determined by the euclidean distance to their neighbors
- Then we craft a loss function between the high dimensional graph and the lower dimensional graph, and do the same-old gradient descent



Following steps

- Similarly, we use matrix algebra to calculate the distance between these 2 matrices as our lost function
- Then we do the gradient descent using SGD algorithm

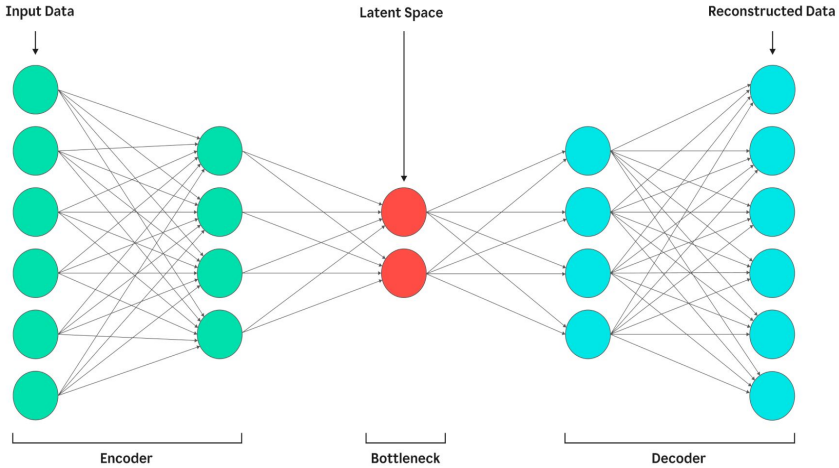
Pros and Cons

- Pros:
  - Very fast compared to the tSNEs
  - Preserves the original data structure compared to tSNE
- Cons:
  - Might be hard to interpret the meaning of the latent dimensions

Auto Encoders

A standard autoencoder is a connected pair of neural networks

- There are encoder and decoder
- The encoder takes the dataset x, converts into a lower dimensional form, z. In a sense the encoding is a compressed technique



Pros and Cons:

- Pros:
  - Also applies to the non-linear data
  - Can be combined with PCA or tSNE
- Cons:
  - Computationally hard, requires neural networks
  - Information can be lost if we only use 2 bottlenecks

Datasets: RNA - seq data

- The dataset comes from the Nature article:

  Shared and distinct transcriptomic cell types across neocortical areas. (Tasic et al. 2018)

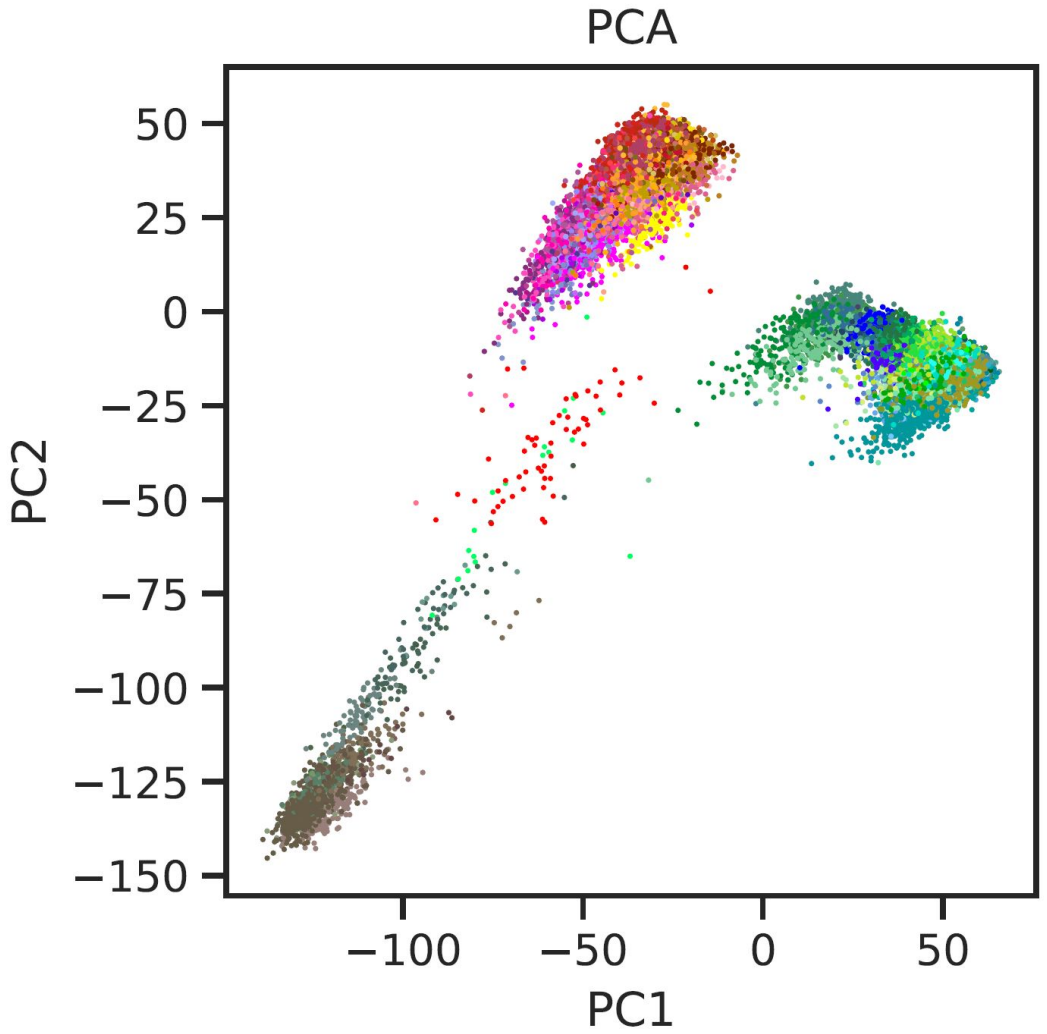
- The cells come from 2 areas at distant poles of mouse neocortex: the primary visual cortex and the anterior lateral cortex: The primary visual cortex and the anterior lateral motor cortex
- 133 distinct cell types identified by the original paper

Dimension of the Original Data:

23822 cells, 3000 genes

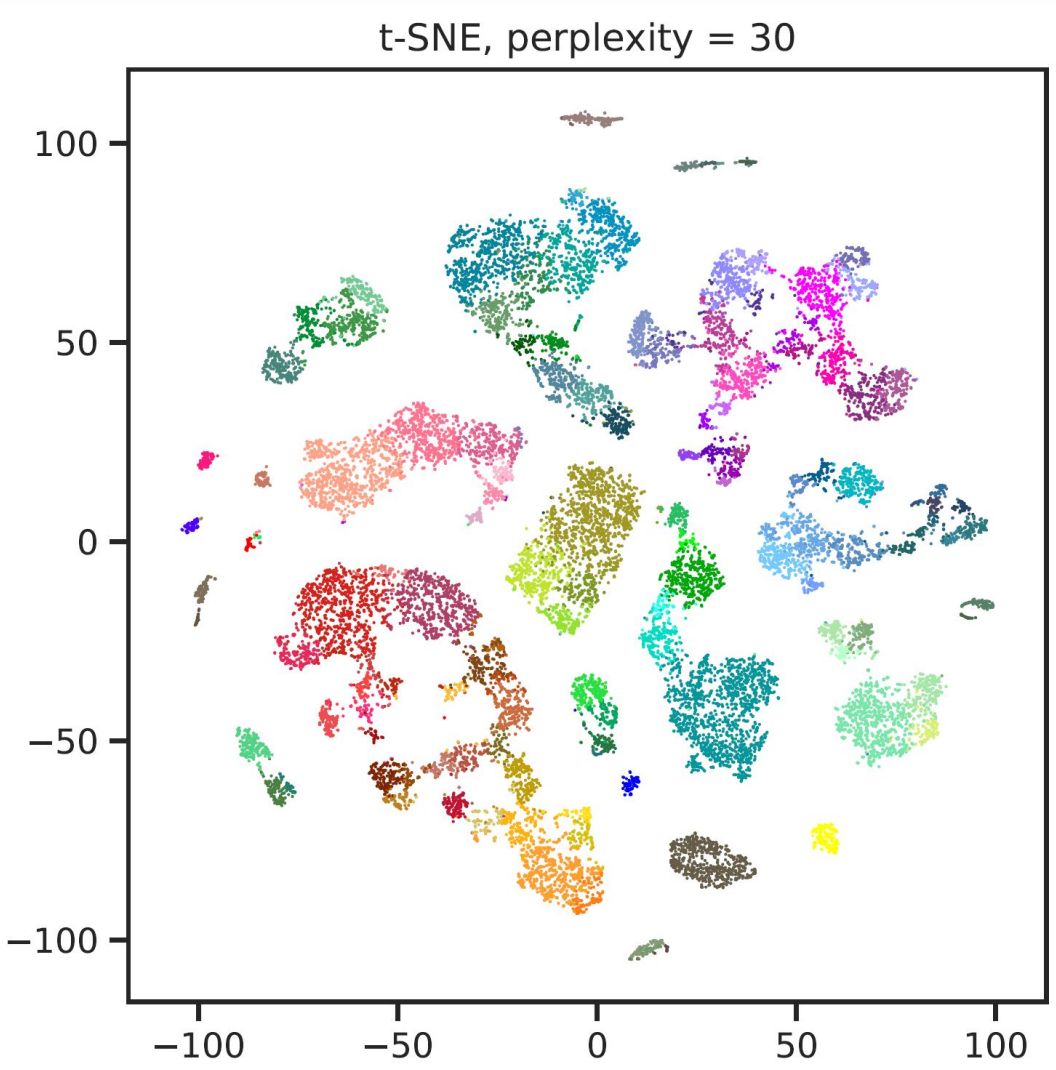
50 PCs are selected from the data

The plot was generated using the first 2 PC



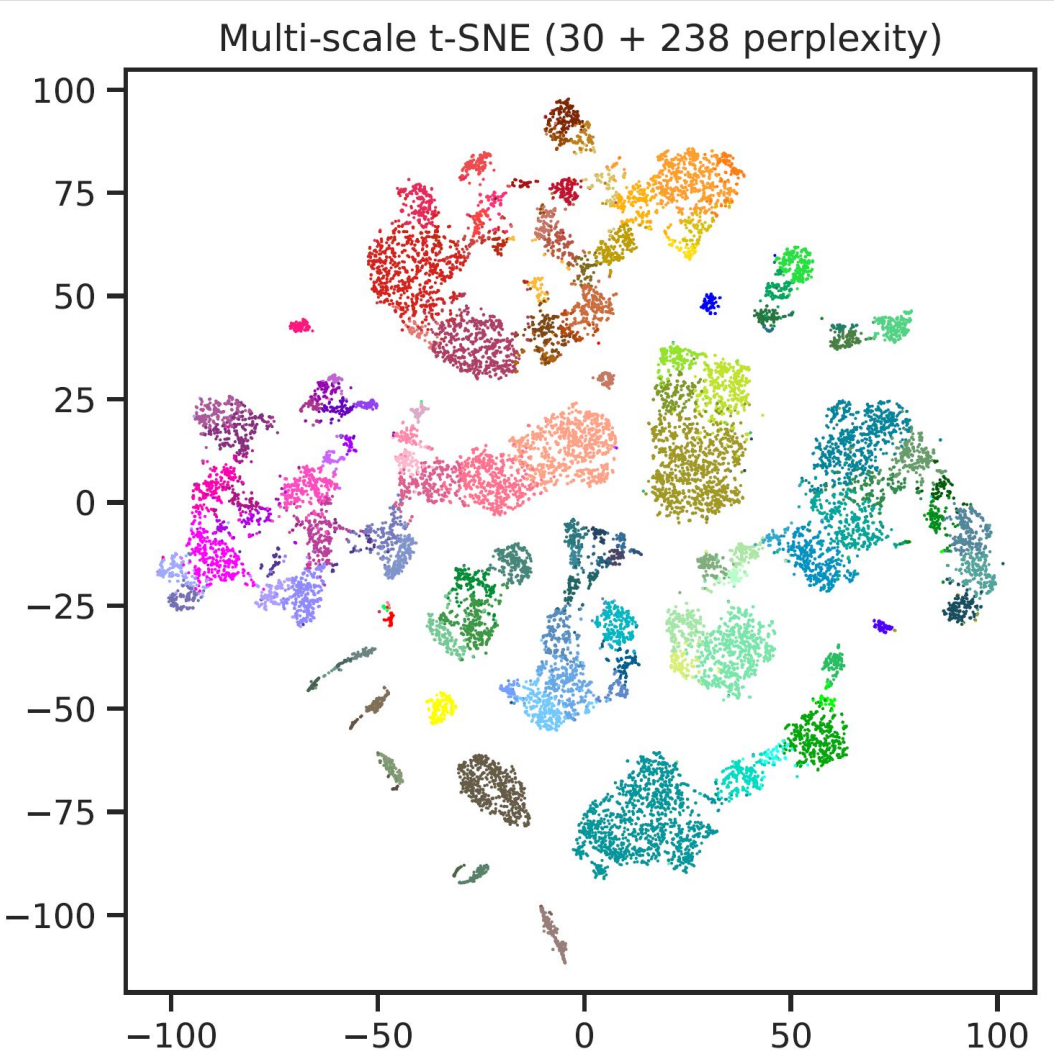
In order to further visualize the data, we used the t-SNE on the 50 PCs we just generated

The hyperparameter, perplexity we chose is 30 for this case



This data is generated using a more advanced tSNE package.

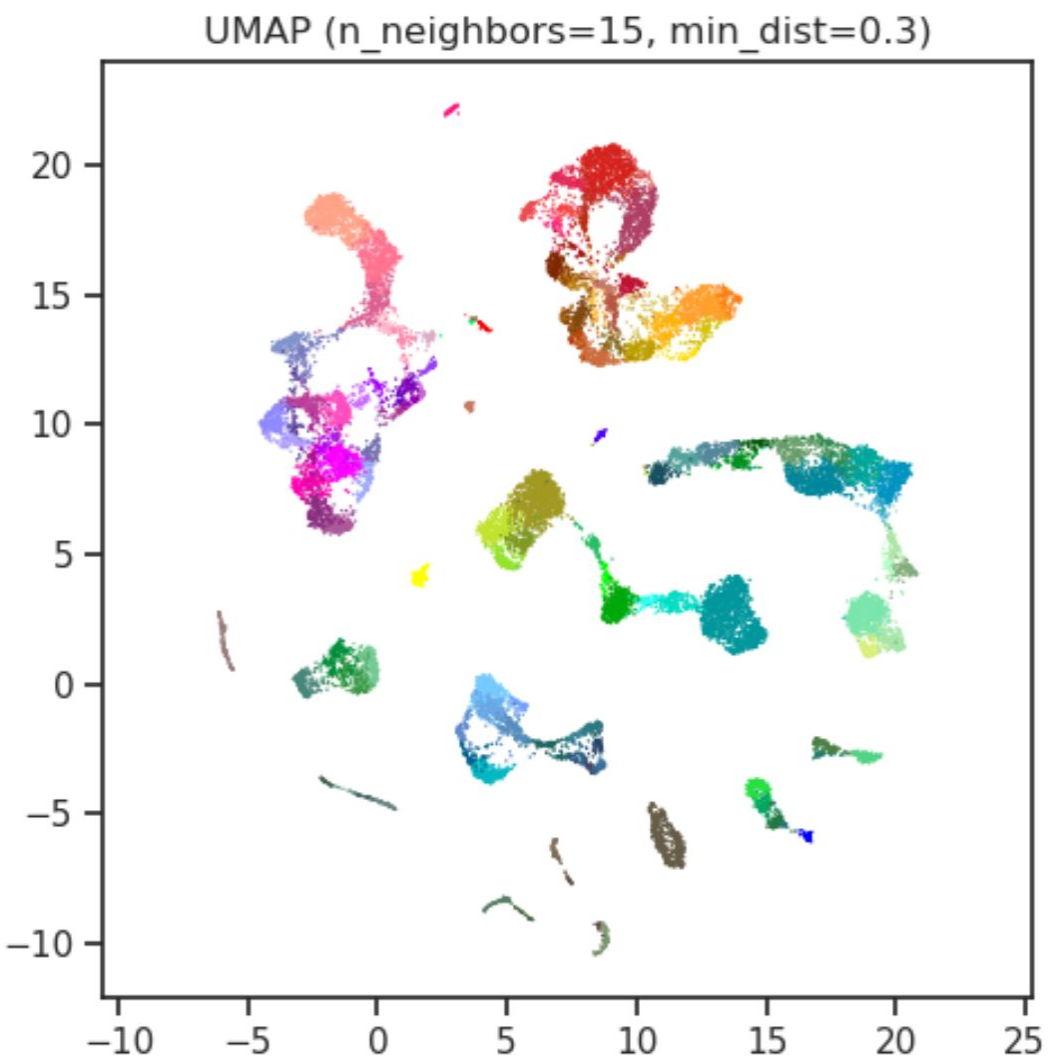
The range of hyperparameter that we can select is larger



The UMAP is applied also on the first 50 PCs we generated from the data

There are 2 hyperparameters.

We consider the closest 15 points as neighbors, not assigning weight to other points

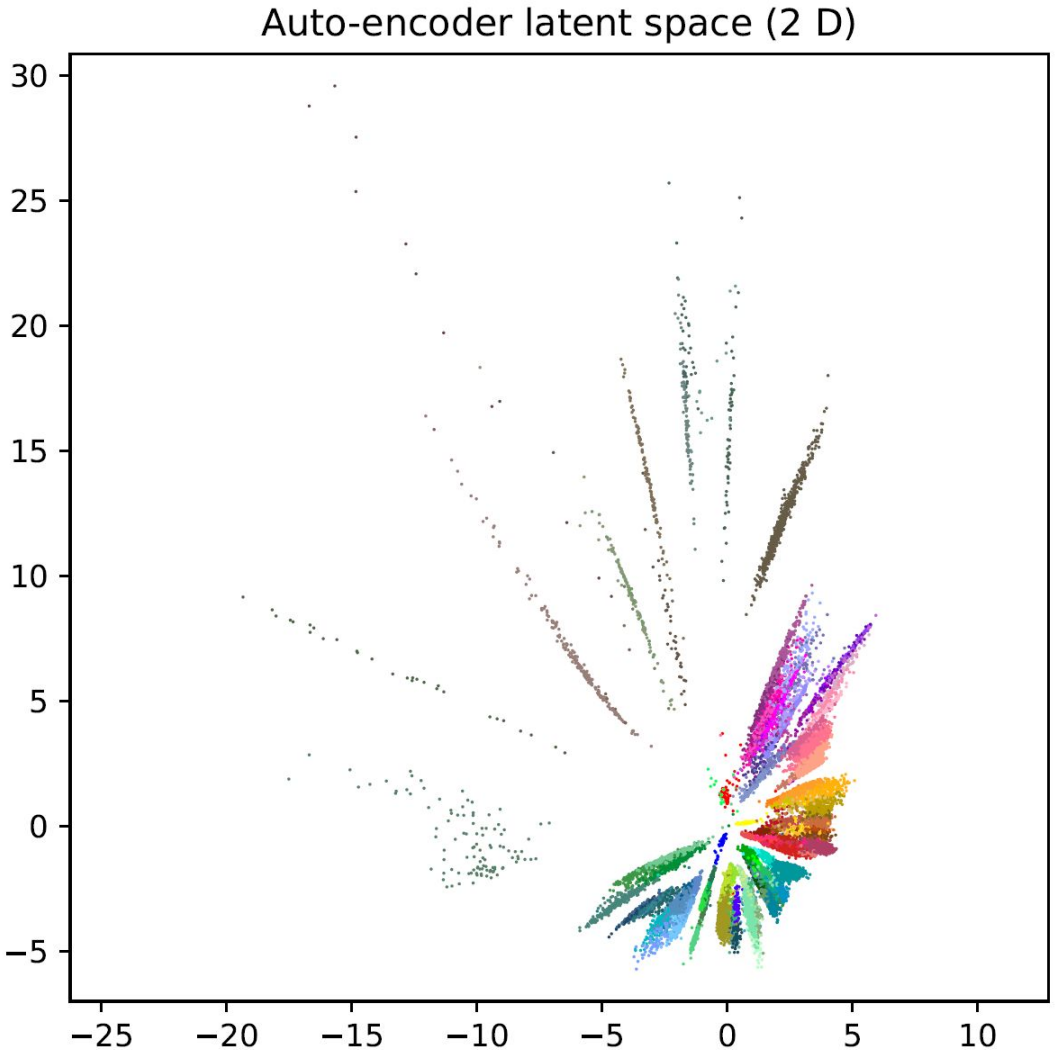


This is the auto-encoder trained on 3000 dimension of the input data

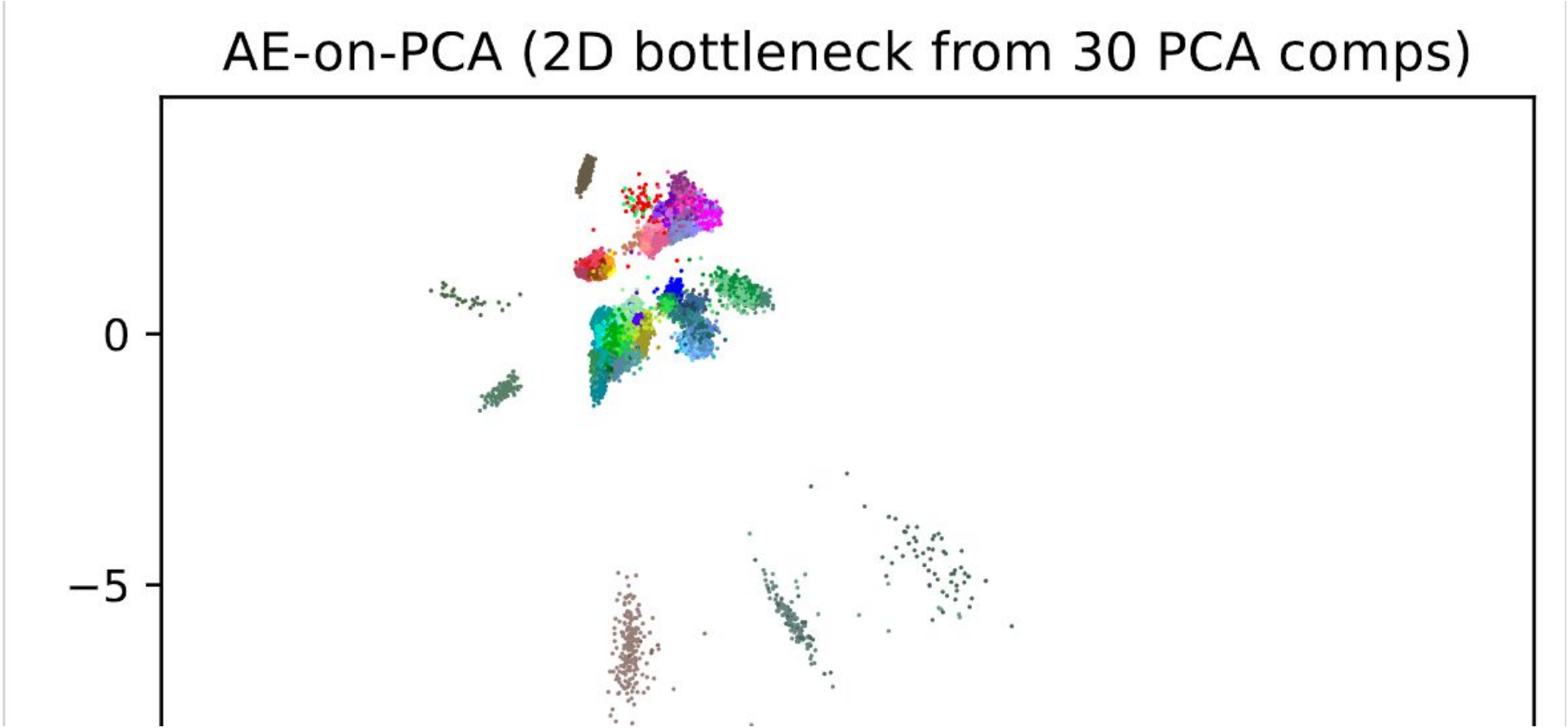
Structure: 3000 > 1024 > 64 > 2 > 64 > 1024 > 3000

The first dimension is the number of the first neuron in the bottle neck layer

The second dimension is the embedding of the second neuron in the bottle neck layer



Moreover we can combine PCA and autoencoders (or any technique we mentioned before)



The curse of dimensionality

If n << p, we will have a problem called the curse of dimensionality

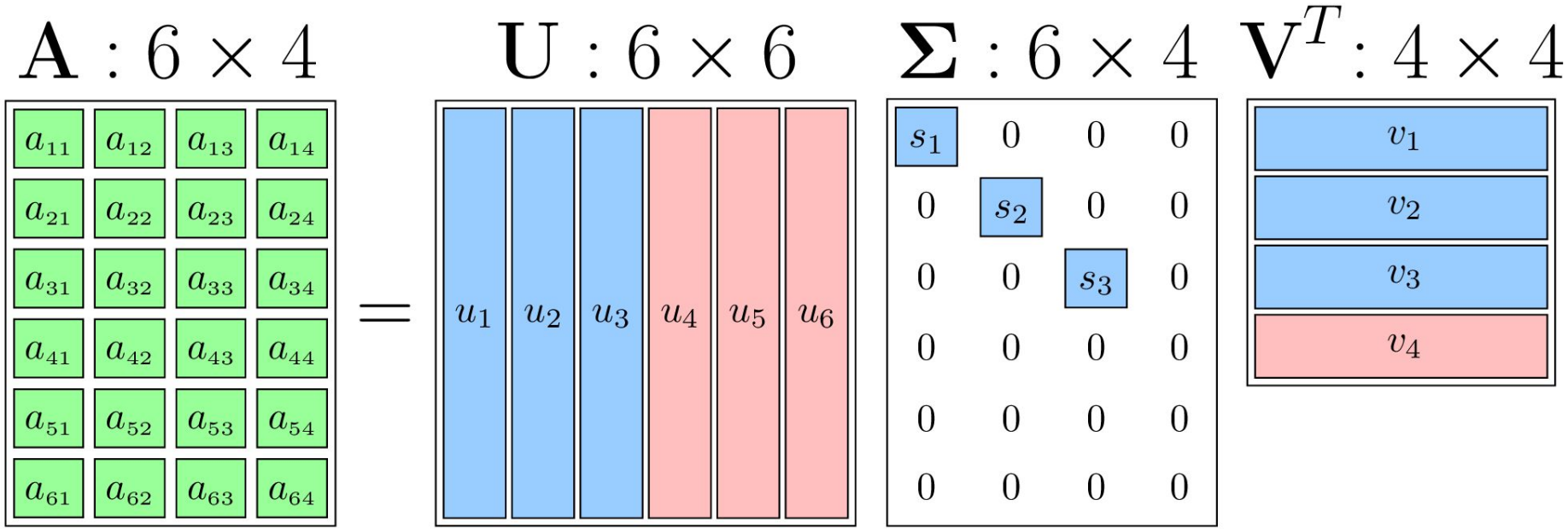
$$\text{If } n \ll p, \text{ then } \det(A^T A) = 0, \text{ which means that we cannot solve the equation } Ax = b$$

Therefore, we must find a way to reduce the number of features in our data set.

$$n > p \qquad n = p \qquad n < p$$



The rationale behind the PCA



1. **Initialize Low-Dimensional Points**: Randomly initialize points $\mathbf{y}_i \in R^d$ where $d \ll D$ (typically $d = 2$ or $d = 3$).

2. **High-Dimensional Probability Distribution**: Convert high-dimensional points $\mathbf{x}_i \in R^D$ into probability distribution using Gaussian similarities:

$$p_{j|i} = \frac{\exp(-\|\mathbf{x}_i - \mathbf{x}_j\|^2/2\sigma_i^2)}{\sum_{k\neq i}\exp(-\|\mathbf{x}_i - \mathbf{x}_k\|^2/2\sigma_i^2)}$$

Symmetrized joint probabilities: $p_{ij} = \frac{p_{j|i}+p_{i|j}}{2n}$

3. **Low-Dimensional Probability Distribution**: Convert low-dimensional points into probability distribution using Student's t-distribution:

$$q_{ij} = \frac{(1 + \|\mathbf{y}_i - \mathbf{y}_j\|^2)^{-1}}{\sum_{k\neq l}(1 + \|\mathbf{y}_k - \mathbf{y}_l\|^2)^{-1}}$$

4. **Loss Function**: Minimize Kullback-Leibler divergence between distributions:

$$C = KL(P\|Q) = \sum_{i,j} p_{ij} \log \frac{p_{ij}}{q_{ij}}$$

5. **Gradient Descent**: Update low-dimensional points using gradient:

$$\frac{\partial C}{\partial \mathbf{y}_i} = 4 \sum_j (p_{ij} - q_{ij})(\mathbf{y}_i - \mathbf{y}_j)(1 + \|\mathbf{y}_i - \mathbf{y}_j\|^2)^{-1}$$

The algorithm iteratively adjusts $\mathbf{y}_i$ to minimize the divergence between P and Q, preserving local neighborhood structure in the embedding.

